Day 3 Task Workflow — Thursday, Feb 27, 2026

> **Sprint:** 1 (Foundation) | **Phase:** Authentication & API Structure
> **Owner:** Lead Architect | **Status:** ⚪ Ready to Start
> **Goal:** Complete JWT authentication system with RBAC, auth dependencies, and core API router structure

---

Day 3 Objective

Complete the authentication and authorization layer so that by end of day:

- Auth service: register, login, refresh token, logout (all functional)
- JWT middleware with access/refresh token flow (15 min access, 7 day refresh)
- RBAC decorators: `require_role(["admin", "analyst"])` working
- Auth dependencies for protected endpoints
- Password hashing with bcrypt (salt rounds=12)
- All API route modules properly structured and mounted
- Health check endpoint fully functional

Scope Adherence Checklist

> **STRICT RULE:** Every task below MUST adhere to the master documentation specifications. No features outside the defined scope.

Requirement	Source Document	Section
JWT auth (access + refresh)	MASTER_DOC_PART2	§7.1
RBAC roles: admin, soc_manager, analyst, viewer	MASTER_DOC_PART2	§7.1
bcrypt password hashing (salt rounds=12)	MASTER_DOC_PART2	§7.1
Bearer token on all endpoints except /auth/login	MASTER_DOC_PART2	§7.1
Role permissions matrix	MASTER_DOC_PART2	§7.2
Type hints, Pydantic models, async/await	GLOBAL_CONTEXT	§9
Production-quality code	GLOBAL_CONTEXT	§7

Task Breakdown

TASK 1 — Auth Service Implementation 🔨

Time Est: 60 min | **Priority:** 🔴 Critical

Create the authentication service with register, login, refresh, and logout functionality.

1.1 Auth Service (`backend/app/services/auth_service.py`)

Function	Purpose	Input	Output
`register()`	Create new user	`UserCreate` schema	`UserResponse`
`login()`	Authenticate user	`UserLogin` schema	`TokenResponse`
`refresh()`	Refresh access token	`TokenRefresh` schema	`TokenResponse`
`logout()`	Invalidate session	user_id	Success message
`get_current_user()`	Get user from token	JWT token	`User` model
`hash_password()`	Hash password with bcrypt	plain password	hashed password
`verify_password()`	Verify password	plain + hash	boolean

Key Implementation Points:

- Use `passlib[bcrypt]` for password hashing (already in requirements.txt)
- Use `python-jose[cryptography]` for JWT encoding/decoding (already in requirements.txt)
- Access token expiry: 15 minutes (from config)
- Refresh token expiry: 7 days (from config)
- JWT secret from `settings.JWT_SECRET`
- JWT algorithm: HS256 (from config)

Verification:

```
cd backend && python -c "from app.services.auth_service import AuthService;
print('[OK] AuthService imported')"
```

TASK 2 — JWT Token Management 🔨

Time Est: 30 min | **Priority:** 🔴 Critical

Implement JWT token creation and validation utilities.

2.1 Token Utilities (`backend/app/services/auth_service.py` or separate `backend/app/core/security.py`)

Function	Purpose	Parameters
`create_access_token()`	Generate access token	`data: dict`, `expires_delta: timedelta`
`create_refresh_token()`	Generate refresh token	`data: dict`, `expires_delta: timedelta`
`decode_token()`	Decode and validate token	`token: str`

Function	Purpose	Parameters
`get_token_payload()`	Extract payload without validation	`token: str`

Token Payload Structure:

```
{
    "sub": "user_uuid",      # Subject (user ID)
    "email": "user@email",   # User email
    "role": "analyst",       # User role
    "type": "access",        # Token type (access/refresh)
    "exp": 1234567890,       # Expiration timestamp
    "iat": 1234567890,       # Issued at timestamp
}
```

Verification:

```
cd backend && python -c "
from app.services.auth_service import create_access_token, decode_token
token = create_access_token({'sub': 'test', 'role': 'analyst'})
payload = decode_token(token)
print('[OK] JWT token created and decoded:', payload.get('sub'))
"
```

TASK 3 — Auth Dependencies 🔨

Time Est: 30 min | **Priority:** ⬤ Critical

Create FastAPI dependencies for authentication and authorization.

3.1 Auth Dependencies (`backend/app/dependencies.py`)

Dependency	Purpose	Usage
`get_current_user()`	Extract and validate user from JWT	`Depends(get_current_user)`
`get_current_active_user()`	Ensure user is active	`Depends(get_current_active_user)`
`require_role(roles)`	RBAC role check decorator	`Depends(require_role(["admin"]))`

Implementation Pattern:

```
from fastapi import Depends, HTTPException, status
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
```

```python
security = HTTPBearer()

async def get_current_user(
    credentials: HTTPAuthorizationCredentials = Depends(security),
    db: AsyncSession = Depends(get_db),
) -> User:
    """Extract and validate user from JWT token."""
    token = credentials.credentials
    payload = decode_token(token)
    if payload is None:
        raise HTTPException(status_code=401, detail="Invalid token")
    user = await get_user_by_id(db, payload["sub"])
    if user is None:
        raise HTTPException(status_code=401, detail="User not found")
    return user

def require_role(allowed_roles: list[str]):
    """Dependency factory for role-based access control."""
    async def role_checker(
        current_user: User = Depends(get_current_user),
    ) -> User:
        if current_user.role not in allowed_roles:
            raise HTTPException(
                status_code=status.HTTP_403_FORBIDDEN,
                detail=f"Role '{current_user.role}' not authorized"
            )
        return current_user
    return role_checker
```

Verification:

```bash
cd backend && python -c "from app.dependencies import get_current_user,
require_role; print('[OK] Auth dependencies imported')"
```

TASK 4 — Auth API Endpoints 🔨

Time Est: 30 min | **Priority:** 🔴 Critical

Update the auth API endpoints to use the auth service.

4.1 Auth Routes (`backend/app/api/v1/auth.py`)

Method	Endpoint	Auth Required	Description
POST	`/auth/register`	Admin only	Create new user account
POST	`/auth/login`	No	Login and get token pair
POST	`/auth/refresh`	Refresh token	Get new access token

Method	Endpoint	Auth Required	Description
GET	`/auth/me`	Yes	Get current user profile
POST	`/auth/logout`	Yes	Logout (invalidate token)

Endpoint Specifications:

POST `/auth/register`

- Request: `UserCreate` schema (email, password, full_name)
- Response: `UserResponse` schema
- RBAC: `require_role(["admin"])`
- Validation: Email uniqueness, password strength (min 8 chars)

POST `/auth/login`

- Request: `UserLogin` schema (email, password)
- Response: `TokenResponse` schema (access_token, refresh_token, expires_in)
- No auth required
- Verify password with bcrypt

POST `/auth/refresh`

- Request: `TokenRefresh` schema (refresh_token)
- Response: `TokenResponse` schema
- Validate refresh token is not expired
- Issue new access token

GET `/auth/me`

- Response: `UserResponse` schema
- Auth required: `Depends(get_current_user)`

Verification:

```
cd backend && python -c "from app.api.v1.auth import router; print('[OK] Auth
router imported, endpoints:', [r.path for r in router.routes])"
```

TASK 5 — API Router Structure 🔨

Time Est: 20 min | **Priority:** 🔴 Critical

Ensure all API route modules are properly structured and mounted.

5.1 API Router Aggregator (`backend/app/api/v1/__init__.py`)

Mount all route modules:

```
from fastapi import APIRouter
from app.api.v1 import auth, system, flows, alerts

api_router = APIRouter()

api_router.include_router(auth.router, prefix="/auth", tags=["auth"])
api_router.include_router(system.router, prefix="/system", tags=["system"])
api_router.include_router(flows.router, prefix="/flows", tags=["flows"])
api_router.include_router(alerts.router, prefix="/alerts", tags=["alerts"])
```

5.2 Main App (`backend/app/main.py`)

Ensure the main app mounts the API router:

```
from app.api.v1 import api_router

app.include_router(api_router, prefix=settings.API_V1_PREFIX)
```

Verification:

```
cd backend && python -c "
from app.main import app
routes = [r.path for r in app.routes if hasattr(r, 'path')]
print('[OK] API routes mounted:', len(routes), 'endpoints')
"
```

TASK 6 — Password Hashing Verification 🔨

Time Est: 10 min | **Priority:** 🟢 Medium

Verify bcrypt password hashing is working correctly.

6.1 Password Hash Test

Verification Script:

```
cd backend && python -c "
from passlib.context import CryptContext

pwd_context = CryptContext(schemes=['bcrypt'], deprecated='auto')

# Test hashing
password = 'TestPassword123!'
hashed = pwd_context.hash(password)
print('[OK] Password hashed:', hashed[:20] + '...')
```

```
# Test verification
is_valid = pwd_context.verify(password, hashed)
print('[OK] Password verified:', is_valid)

# Test wrong password
is_invalid = pwd_context.verify('WrongPassword', hashed)
print('[OK] Wrong password rejected:', not is_invalid)
"
```

Expected Output:

```
[OK] Password hashed: $2b$12$...
[OK] Password verified: True
[OK] Wrong password rejected: True
```

TASK 7 — Type Checking & Linting 🔨

Time Est: 10 min | **Priority:** 🟡 Medium

Ensure all new code passes type checking and linting.

Check	Command	Expected
Lint	`cd backend && ruff check app/services app/dependencies.py app/api/v1/auth.py`	No errors
Type check	`cd backend && mypy app/services app/dependencies.py app/api/v1/auth.py --ignore-missing-imports`	No errors
Format	`cd backend && ruff format app/services app/dependencies.py app/api/v1/auth.py`	Code formatted

Verification:

```
cd backend && ruff check app/services app/dependencies.py app/api/v1/auth.py &&
mypy app/services app/dependencies.py app/api/v1/auth.py --ignore-missing-imports
&& echo "[OK] All checks passed"
```

Files Created/Modified Today

```
threatmatrix-ai/
├── backend/
│   ├── app/
│   │   ├── services/
```

```
│   │   │   ├── __init__.py          🔨 Task 1
│   │   │   └── auth_service.py      🔨 Task 1, 2
│   │   ├── dependencies.py          🔨 Task 3
│   │   ├── api/v1/
│   │   │   ├── __init__.py          🔨 Task 5 (update)
│   │   │   └── auth.py              🔨 Task 4 (update)
│   │   └── main.py                  🔨 Task 5 (verify mount)
```

Verification Checklist

> **Every item below MUST be verified before marking task complete.**

#	Verification	Command	Expected Result
1	AuthService imports	`python -c "from app.services.auth_service import AuthService"`	No errors
2	JWT token creation	`python -c "from app.services.auth_service import create_access_token"`	No errors
3	Auth dependencies	`python -c "from app.dependencies import get_current_user, require_role"`	No errors
4	Auth router	`python -c "from app.api.v1.auth import router"`	No errors
5	Password hashing	Test bcrypt hash + verify	True
6	API routes mounted	Check app.routes	All endpoints visible
7	Linting	`ruff check`	No errors
8	Type checking	`mypy`	No errors

Scope Adherence Verification

Requirement	Source	Verification
JWT auth (access + refresh)	MASTER_DOC_PART2 §7.1	Check token creation with 15min/7day expiry
RBAC roles: admin, soc_manager, analyst, viewer	MASTER_DOC_PART2 §7.1	Check require_role decorator
bcrypt password hashing (salt rounds=12)	MASTER_DOC_PART2 §7.1	Check CryptContext config
Bearer token on all endpoints except /auth/login	MASTER_DOC_PART2 §7.1	Check HTTPBearer security

Requirement	Source	Verification
Role permissions matrix	MASTER_DOC_PART2 §7.2	Verify role checks match matrix
Type hints everywhere	GLOBAL_CONTEXT §9	Run mypy
Production-quality code	GLOBAL_CONTEXT §7	Run ruff + mypy

Blockers

Blocker	Severity	Mitigation	Status
Docker not running (Windows update corruption)	⚪ Medium	Code-first approach, verify when Docker fixed	Mitigated
Cannot test endpoints without running server	⚪ Medium	Unit test auth functions, integration test when Docker fixed	Mitigated

Tomorrow's Preview (Day 4)

- Redis setup + pub/sub test
- Next.js frontend shell: sidebar navigation
- All 10 module pages created (stub pages)
- Navigation working with active state highlighting

Task workflow for Day 3 — ThreatMatrix AI Sprint 1